技术部第四次技术分享会

基础数据结构与 Java 容器

姚雨森　技术部分管主席

回顾数组：长度需要提前确定

数组把一组同类型的数据放在一起，通过从 0 开始的下标访问。

```
int[] prices = new int[3];
prices[0] = 12;
prices[1] = 8;
prices[2] = 25;
System.out.println(prices.length); // 3
```

- 数组一旦创建，长度就固定了；可以修改元素，不能直接增加位置。
- 预留太少，后续数据放不下；预留太多，又有空余位置需要管理。
- 商品不断增加或下架时，我们希望能直接添加、删除元素，让保存的数据数量随需求变化。

除了位置编号，我们还需要数据的标签

数组用下标表示**位置**，但我们有时更想用一个**标签**找到数据。

- 按位置访问：第 0 件商品的价格是多少？
- 按标签查找：商品编号 P1001 对应的价格是多少？
- 这种用于查找的标签称为键（key），对应的数据称为值（value）。

数组也能配合查找代码完成任务，但我们希望有更直接的存储方式：

- **数量可变的列表**：方便添加、删除元素，不必自己安排数组长度。
- **键与值的映射**：根据商品编号等标签，保存和查询对应的数据。

两种定位方式：索引与键

本次学习两类容器，它们都存储数据，但定位数据的方式不同。

	List：列表	**Map：映射**
定位依据	索引（下标），表示位置	键，表示数据的标签
例子	第 0 件商品	编号 P1001 的商品
本次使用	`ArrayList`	`HashMap`
其他实现	`LinkedList`	`TreeMap` / `LinkedHashMap`

`List`、`Map` 是**接口**，约定操作；具体的类提供**实现**。

同一接口可以有多种实现，操作约定相通，但顺序、性能等行为可能不同。

容器：存储数据与增删查改

容器都用于组织和存储数据，区别在于数据的组织方式和操作行为。

- **增**：加入新的数据。
- **删**：移除不再需要的数据。
- **查**：读取数据，或判断数据是否存在。
- **改**：更新已经保存的数据。

学习每个容器时，先问**怎样组织和定位数据**，再看**如何增删查改**。
这些是理解操作的共同问题，不保证每个容器都有四个独立的方法。

ArrayList：按下标组织元素序列

可以把 ArrayList 理解为可以变化长度的数组，直接添加或删除元素。

```
// import java.util.ArrayList;
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
System.out.println(products.size()); // 2
```

方法签名：boolean add(E element)、int size()。

E 代表列表的元素类型；方法名前是返回类型，括号内是参数。

下标仍从 0 开始；元素数量用 size() 获取。

ArrayList 的增：追加或指定位置插入

`boolean add(E element)`：追加到末尾，成功后返回 `true`。
`void add(int index, E element)`：在指定位置插入，无返回值。

`E` 表示元素类型；本例使用 `String`。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Bread");
products.add(1, "Milk");
System.out.println(products); // [Apple, Milk, Bread]
```

- 插入后，原位置及后面的元素向后移动，数量加一。
- 插入位置允许从 `0` 到 `size()`；`size()` 表示末尾。
- `add` 新增元素，`set` 替换已有元素。

ArrayList 的查与改：get 和 set

`E get(int index)`：返回指定位置的元素。
`E set(int index, E element)`：替换元素，返回原来的元素。

数组：用下标读取和赋值

```
String[] products = {"Apple"};

String first = products[0];

products[0] = "Bread";
```

ArrayList：调用对应的方法

```
ArrayList<String> products =
    new ArrayList<>();
products.add("Apple");
String first = products.get(0);
products.set(0, "Bread");
```

- `get(i)` 对应数组的 `products[i]`：读取已有位置。
- `set(i, value)` 对应数组的下标赋值：替换元素，数量不变。

空列表要先用 `add` 添加元素，不能用 `set(0, ...)` 创建第一个元素。

尖括号指定列表中的元素类型

ArrayList<String> 表示这个列表用来存放 String 类型的元素。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
String first = products.get(0);
// products.add(12); // Type mismatch
```

- 尖括号中的类型叫类型参数，这种写法称为泛型。
- 必须填写**引用类型**，例如 String、Student 等类。
- int、double 等**基本类型**不能直接填进去。

```
// ArrayList<int> prices = new ArrayList<>(); // Invalid
```

`boolean add(String element)` `String get(int index)`

存放整数时，使用 Integer

整数列表使用 ArrayList<Integer>。

`boolean add(Integer element)` `Integer get(int index)`

```
ArrayList<Integer> prices = new ArrayList<>();
prices.add(12);
int price = prices.get(0);
```

基本类型	对应的类	基本类型	对应的类
byte	Byte	float	Float
short	Short	double	Double
int	Integer	char	Character
long	Long	boolean	Boolean

这些对应关系涉及**包装类**，这里先记住用法。

ArrayList 的删：按下标删除

`E remove(int index)`：删除指定位置的元素，并返回被删元素。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
products.add("Bread");
products.remove(1);
System.out.println(products);         // [Apple, Bread]
System.out.println(products.get(1)); // Bread
System.out.println(products.size()); // 2
```

- 删除前，Milk 的下标是 1，Bread 的下标是 2。
- 删除后，Bread 的下标变为 1，不会留下空位。

ArrayList 的删：按值删除第一个匹配项

`boolean remove(Object o)`：按值删除，返回是否删除成功。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
products.add("Apple");
boolean removed = products.remove("Apple");
System.out.println(removed);  // true
System.out.println(products); // [Milk, Apple]
boolean found = products.remove("Tea");
System.out.println(found);    // false
```

- 有重复元素时，只删除从前往后找到的第一个。
- 找到并删除返回 `true`；没有找到返回 `false`，列表不变。

ArrayList 的删：区分整数下标与数值

`E remove(int index)`：按下标删除。
`boolean remove(Object o)`：按值删除。

```
ArrayList<Integer> prices = new ArrayList<>();
prices.add(12);
prices.add(8);
prices.add(1);
prices.remove(1);
System.out.println(prices); // [12, 1]
prices.remove(Integer.valueOf(1));
System.out.println(prices); // [12]
```

- `remove(1)`：删除下标为 1 的元素，本例中是 8。
- `remove(Integer.valueOf(1))`：删除数值为 1 的元素。

ArrayList 的查：是否存在、出现在哪里

`boolean contains(Object o)`：判断是否存在。
`int indexOf(Object o)`：返回首次出现的下标，未找到返回 -1。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
products.add("Apple");
System.out.println(products.contains("Milk")); // true
System.out.println(products.contains("Tea"));  // false
System.out.println(products.indexOf("Apple")); // 0
System.out.println(products.indexOf("Tea"));   // -1
```

- `contains(value)`：只想知道元素是否存在。
- `indexOf(value)`：需要第一个匹配元素的下标。
- `-1` 表示没有找到，不能把它作为下标传给 `get`。

ArrayList 的遍历：通过下标逐个读取

`int size()` 返回元素数量；`E get(int index)` 按下标读取。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
for (int i = 0; i < products.size(); i++) {
    System.out.println(i + ": " + products.get(i));
}
// 0: Apple
// 1: Milk
```

- 数组中的 `length`，在这里换成 `size()`。
- 数组中的 `products[i]`，在这里换成 `products.get(i)`。
- 循环条件是 `i < products.size()`，不能写成小于等于。

增强 for：直接取得每一个元素

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
for (String product : products) {
    System.out.println(product);
}
// Apple
// Milk
```

- 冒号右边是要遍历的列表 products。
- 冒号左边声明变量 product，每轮取得一个元素。
- 按列表顺序访问元素；同样的写法也适用于数组。

增强 for 中，不要直接增删原列表

`boolean remove(Object o)`：按值删除。
遍历时直接增删可能抛出 `ConcurrentModificationException`。

```
// Incorrect: do not modify the list this way.
for (String product : products) {
    if (product.equals("Milk")) {
        products.remove(product);
    }
}
```

- 没有抛出异常，也不能说明这种写法可靠。
- 如果只需删除一个已知值，可以直接调用 `products.remove("Milk")`，不必遍历。
- 需要筛选多个元素时，可以先把要保留的元素放入**另一个列表**。

ArrayList 回顾：按下标完成增删查改

保存有位置顺序的元素，允许重复；get、set 用下标定位。

操作	方法	行为
增	`add`	追加或插入，数量加一
删	`remove`	按下标或值删除
查	`get / contains / indexOf`	读取、判断存在、查位置
改	`set`	替换已有位置，数量不变

遍历让我们逐个读取元素；`size()` 告诉我们保存了多少个元素。

HashMap：存储数据，通过键定位

ArrayList 按**位置**访问元素；但商品编号并不是列表下标。

商品编号：键（key）	价格：值（value）
P1001	12 元
P1002	8 元
P1003	12 元

- 我们想问“P1002 的价格是多少”，而不是“第几个元素是多少”。
- HashMap 同样存储数据，改用键来定位对应的值。
- 每个键只对应一个值；不同的键可以对应相同的值。

创建 HashMap：分别指定键和值的类型

HashMap<K, V>：K 是键的类型，V 是值的类型。

无参构造方法：public HashMap()，创建一个空映射。

```
// import java.util.HashMap;
HashMap<String, Integer> prices = new HashMap<>();
```

- String：键是商品编号，例如 "P1001"。
- Integer：值是整数价格，例如 12；不能填写 int。
- 右边的 <> 由 Java 推断类型，不必重复写一遍。
- 此时映射还是空的；声明类型并不会自动加入商品和价格。

和 ArrayList 一样，键和值的类型参数都必须使用**引用类型**。

HashMap 的增：put 新的键

`V put(K key, V value)`：保存对应关系，返回这个键原来的值。

```
HashMap<String, Integer> prices = new HashMap<>();
Integer old = prices.put("P1001", 12);
prices.put("P1002", 8);
System.out.println(old); // null
```

- 第一个参数是键，第二个参数是值，顺序不能交换。
- 本例中，K 是 `String`，V 是 `Integer`。
- 新键没有旧值，返回 `null`；不需要返回值时可以不接收。

HashMap 的改：put 已有的键

`V put(K key, V value)`；`int size()` 返回键值对的数量。

```
HashMap<String, Integer> prices = new HashMap<>();
prices.put("P1001", 12);
prices.put("P1002", 8);
Integer old = prices.put("P1001", 15);
System.out.println(old);           // 12
System.out.println(prices.size()); // 2
```

- P1001 的价格从 12 变成 15。
- 键不会重复保存，因此键值对数量仍为 2。
- 与列表的 add 不同，put 既可以新增，也可以更新。

HashMap 的查：get 按键读取值

`V get(Object key)`：返回对应的值；键不存在时返回 `null`。

```
HashMap<String, Integer> prices = new HashMap<>();
prices.put("P1001", 12);
Integer price = prices.get("P1001");
Integer missing = prices.get("P9999");
System.out.println(price);   // 12
System.out.println(missing); // null
```

- 传入的是商品编号，不是位置下标。
- `Object key` 表示接收对象；使用时应传入与键类型匹配的值。
- 不要把可能为 `null` 的结果直接赋给 `int`，否则可能出现空指针异常。

HashMap 的查：containsKey 判断键存在

`boolean containsKey(Object key)`：判断是否保存了这个键。

```
HashMap<String, Integer> prices = new HashMap<>();
prices.put("P1001", 12);
String id = "P9999";
if (prices.containsKey(id)) {
    System.out.println(prices.get(id));
} else {
    System.out.println("Product not found");
}
```

- 适合“找到就处理，否则提示不存在”的场景。
- containsKey 判断的是键，不是值。
- HashMap 允许值为 null，所以 get 返回 null 不一定表示键不存在。

HashMap 的删：remove 按键删除

`V remove(Object key)`：删除键值对，返回原来的值。

```
HashMap<String, Integer> prices = new HashMap<>();
prices.put("P1001", 12);
prices.put("P1002", 8);
Integer removed = prices.remove("P1001");
System.out.println(removed);                    // 12
System.out.println(prices.containsKey("P1001")); // false
System.out.println(prices.remove("P9999"));      // null
```

- 传入要删除的键，不是价格，也不是下标。
- 删除后，键和对应的值一起从映射中移除。
- 键不存在时返回 `null`，映射保持不变。

HashMap 的遍历：keySet 读取所有键

`Set<K> keySet()`：返回由所有键组成的集合，可以用增强 `for` 遍历。

```
HashMap<String, Integer> prices = new HashMap<>();
prices.put("P1001", 12);
prices.put("P1002", 8);
for (String id : prices.keySet()) {
    System.out.println(id);
}
```

- 每轮取得一个商品编号；这里的 K 是 `String`。
- 返回类型中的 `Set` 表示不重复元素的集合；这里直接遍历即可。
- HashMap 不保证遍历顺序，不能依赖插入顺序或键的排序。

HashMap 的遍历：entrySet 读取键值对

`Set<Map.Entry<K, V>> entrySet()`
每个 `Map.Entry` 是一个键值对：`K getKey()`、`V getValue()`。

```
// import java.util.Map;
HashMap<String, Integer> prices = new HashMap<>();
prices.put("P1001", 12);
prices.put("P1002", 8);
for (Map.Entry<String, Integer> entry : prices.entrySet()) {
    String id = entry.getKey();
    Integer price = entry.getValue();
    System.out.println(id + ": " + price);
}
```

同时需要商品编号与价格时，用这种写法。

和列表一样，不要在增强 `for` 中直接对原映射新增或删除键值对。

HashMap 回顾：按键完成增删查改

保存键值对应关系；键唯一，值可以重复，通过键定位数据。

操作	方法	行为
增	`put` 新键	新增一组键值对
删	`remove`	按键移除键值对
查	`get` / `containsKey`	读取值、判断键存在
改	`put` 已有键	替换对应的值

`put` 同时承担增和改，具体行为取决于键是否已经存在。

练习：用查、增、改统计词频

给定一组单词，用 HashMap<String, Integer> 保存“单词 → 次数”。

```
String[] words = {"apple", "milk", "apple", "bread", "milk"};
```

- 遍历单词数组，每次处理一个单词。
- **查**：读取已有次数；未出现过时按 0 处理。
- 次数加一，**新增**这个单词，或**修改**它已有的次数。
- 遍历键值对，输出每个单词及其次数，不要求输出顺序。

预期结果：apple 为 2 次，milk 为 2 次，bread 为 1 次。

提示：V getOrDefault(Object key, V defaultValue)、V put(K key, V value)。

词频统计：读取旧次数，再更新

```
String[] words = {"apple", "milk", "apple", "bread", "milk"};
HashMap<String, Integer> counts = new HashMap<>();
for (String word : words) {
    int count = counts.getOrDefault(word, 0);
    counts.put(word, count + 1);
}
for (Map.Entry<String, Integer> entry : counts.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}
```

- 第一次遇到 apple：**查**到默认值 0，**新增**次数 1。
- 第二次遇到 apple：**查**到旧值 1，**修改**次数为 2。

`V getOrDefault(Object key, V defaultValue)` `V put(K key, V value)`

HashMap 查重：把待检查的数据作为键

同一个学号可能重复提交签到，我们希望发现哪些提交是重复的。

- 用学号作为**键**，用第一次出现的位置作为**值**。
- **查**：先用 `containsKey` 判断这个学号是否已经出现。
- 已经出现：报告重复，并用 `get` 读取第一次出现的位置。
- 尚未出现：用 `put` **新增**记录，留给后续查重。

`boolean containsKey(Object key)` 判断是否存在。

`V get(Object key)` 读取旧记录。
`V put(K key, V value)` 保存记录。

签到查重：先查，再决定是否新增

```
String[] ids = {"2026001", "2026002", "2026001"};
HashMap<String, Integer> firstSeen = new HashMap<>();
for (int i = 0; i < ids.length; i++) {
    String id = ids[i];
    if (firstSeen.containsKey(id)) {
        System.out.println(id + " first at " + firstSeen.get(id));
    } else {
        firstSeen.put(id, i);
    }
}
// 2026001 first at 0
```

重复时不覆盖旧值，所以保留的始终是**第一次出现的下标**。

本页方法：containsKey 查存在，get 查位置，put 增记录。

都用于存储数据，组织方式决定操作行为

	List：本次用 ArrayList	Map：本次用 HashMap
组织方式	元素序列	键值对应
定位依据	索引（下标）	键
增	`add`	`put` 新键
删	`remove`	`remove`
查	`get / contains`	`get / containsKey`
改	`set`	`put` 已有键

先确定怎样组织和定位数据，再用增删查改理解方法的作用。

`ArrayList` 和 `HashMap` 各是一种实现，其他实现也遵循对应接口的约定。